Beyond Gradient Descent

Unit 4

MDS 655

The Challenges with Gradient Descent

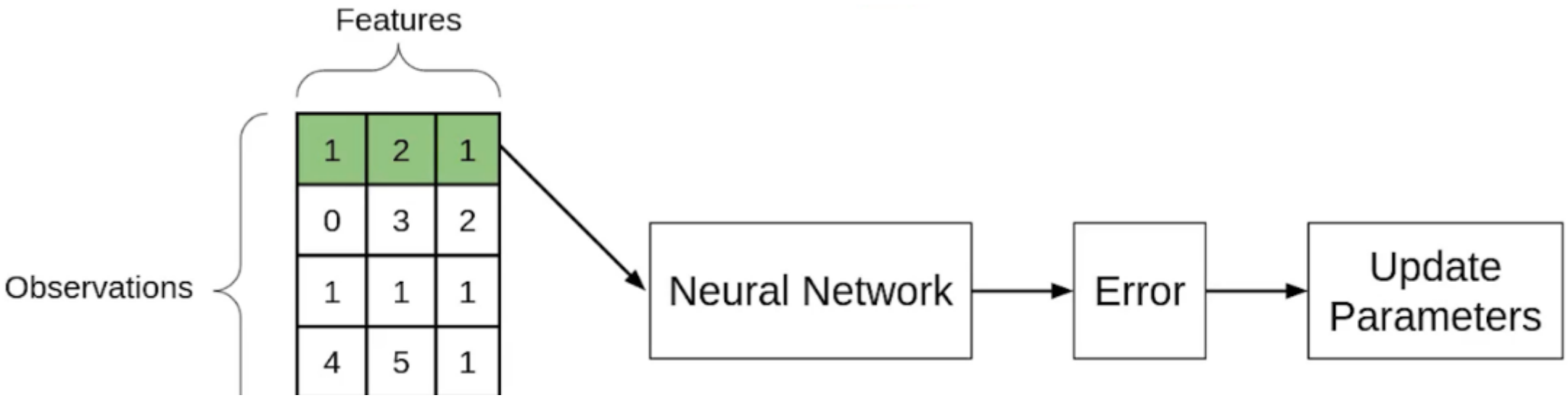
- Local minima
- Vanishing and exploding gradient
- Requires massive labeled datasets (ImageNet, CIFAR…)
- Requires better hardware (GPU)
- Several algorithms needs to be designed

Breakthroughs to tackle the challenges

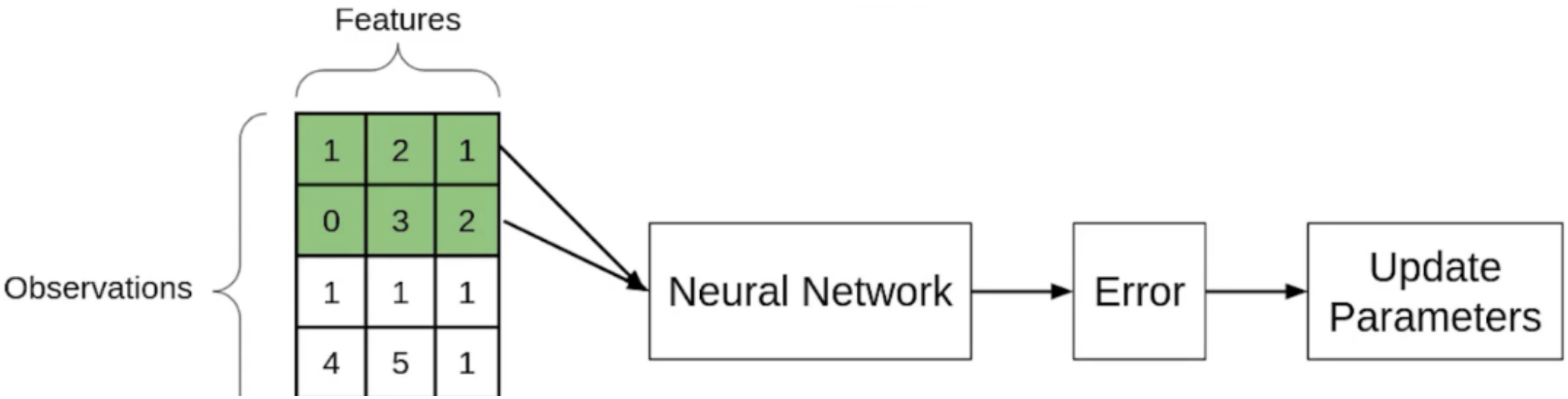
- Layer-wise greedy pre-training
- mini-batch gradient descent
- Training models in an end-to-end fashion
- Nonconvex optimizers

Batch Gradient

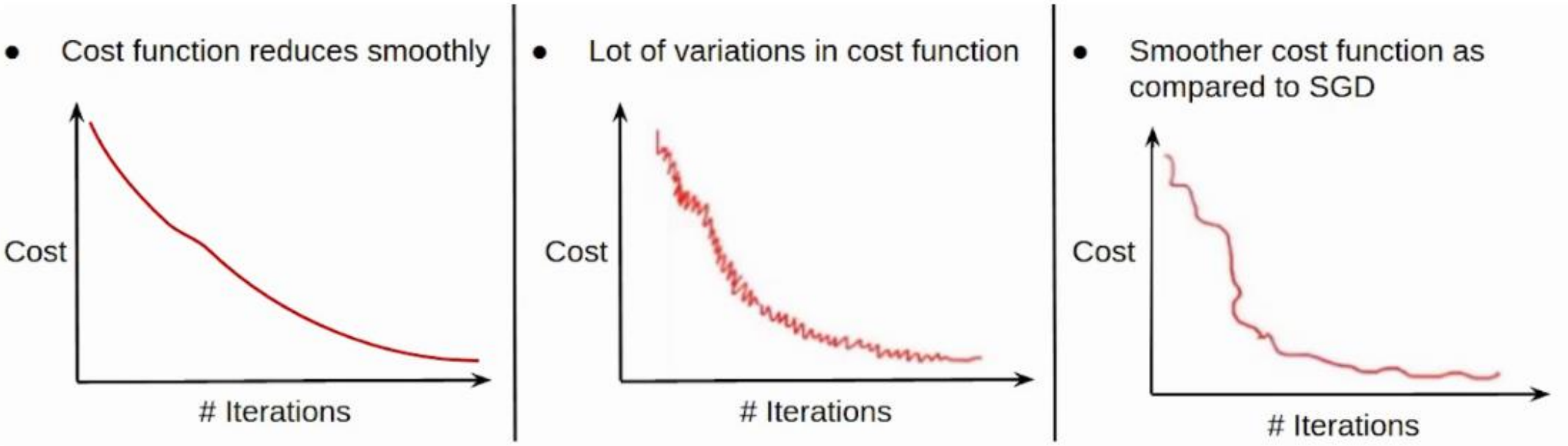
SGD


Features
Observations
1 2 1
0 3 2
1 1 1
4 5 1
Neural Network
Error
Update Parameters

Mini Batch: Batch size=2



Batch/SGD/Mini Batch



Batch Gradient Descent

- Entire dataset for updation
- Cost function reduces smoothly
- Computation cost is very high

Stochastic Gradient Descent (SGD)

- Single observation for updation
- Lot of variations in cost function
- Computation time is more

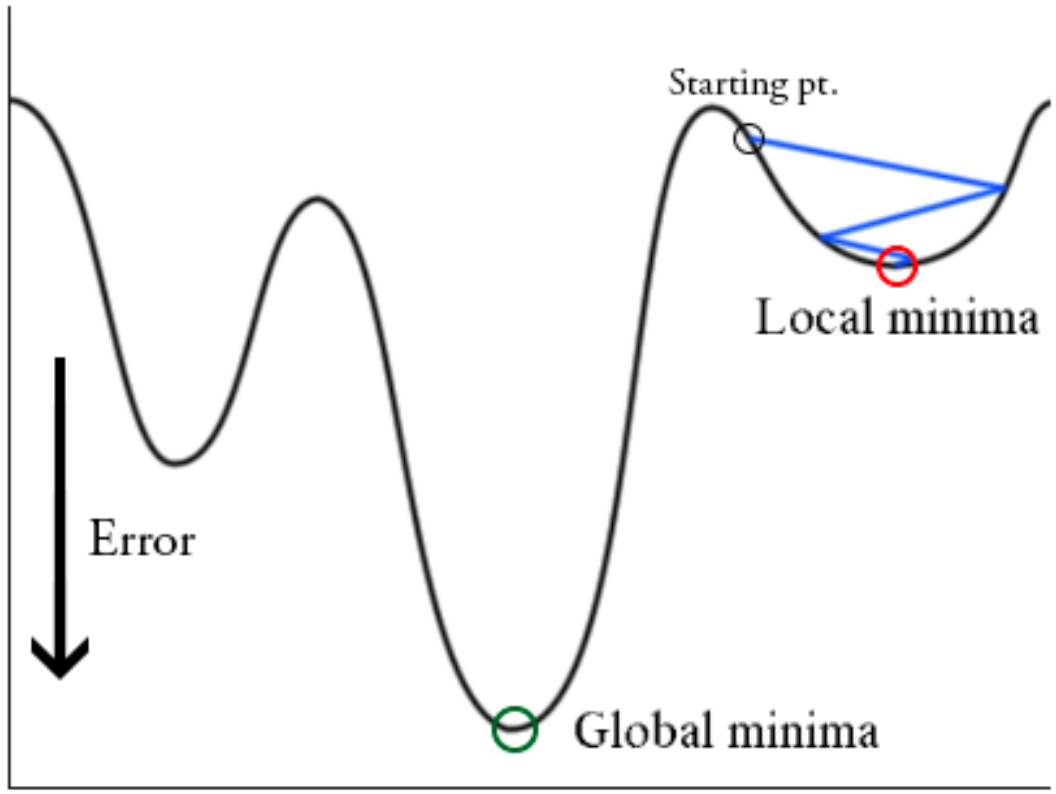
Mini-Batch Gradient Descent

- Subset of data for updation
- Smoother cost function as compared to SGD
- Computation time is lesser than SGD
- Computation cost is lesser than Batch Gradient Descent

Local Minima in the Error Surfaces of Deep Networks

- With minimal local information inferring global structure of the error surface is a challenge
- Correspondence between local and global structure is very little

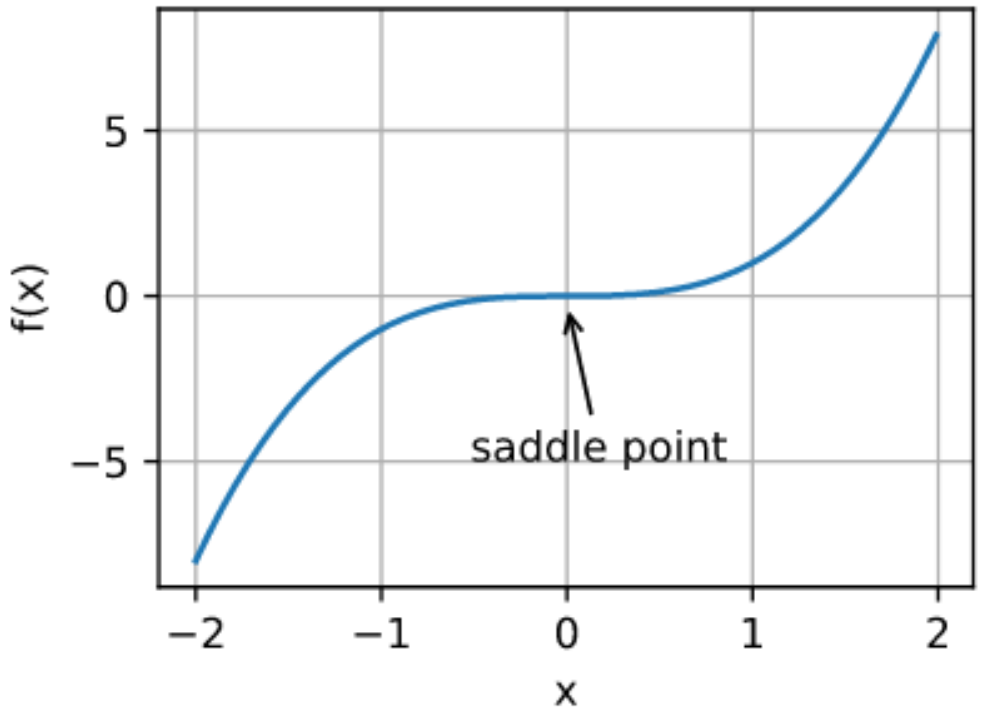
- Finding global minima is easy
  - Convex error surface
- Finding global minima is difficult
  - Non-Convex error surface



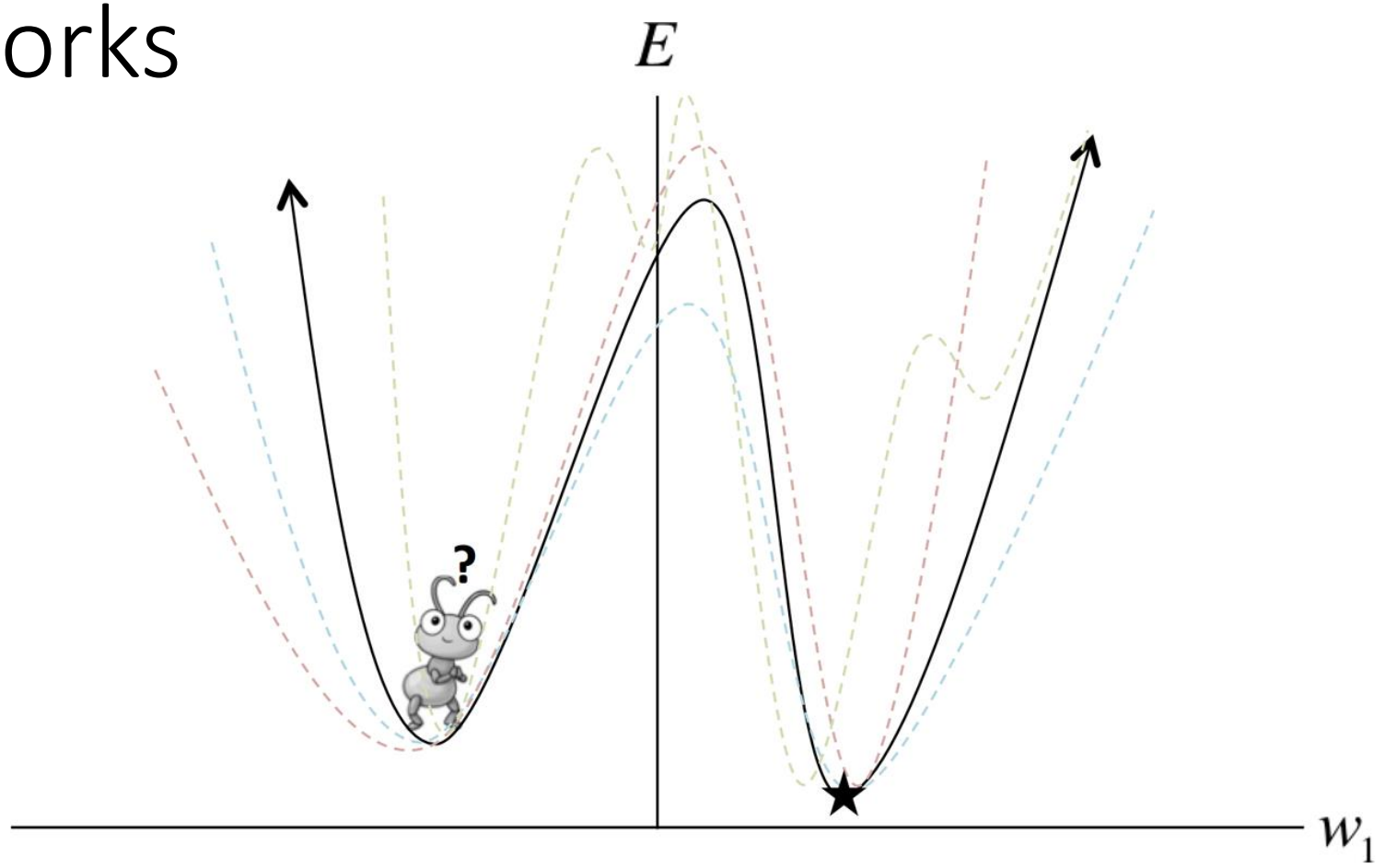
- Convergence point of gradient descent depends on the starting point and learning rate
- Slope of cost functions is very close to zero at local minima and thus model stops learning

Saddle point

- points where the function attains neither a local maximum value nor a local minimum value
- Slope at this point is also very near to zero and thus model stops learning



Local Minima in the Error Surfaces of Deep Networks



Mini-batch gradient descent may aid in escaping shallow local minima

but

often fails when dealing with deep local minima

Vanishing and exploding gradient

- If Partial derivative are large
  - Gradient will increase exponentially
  - Back propagation with lots of epochs will cause problem of exploding gradient
  - Eventually will overshoot the minima
- If Partial derivative are small
  - Gradient will decrease exponentially
  - Back propagation with lots of epochs will cause problem of vanishing gradient
  - Eventually causes the weights update unchanged

Model Indentifiability

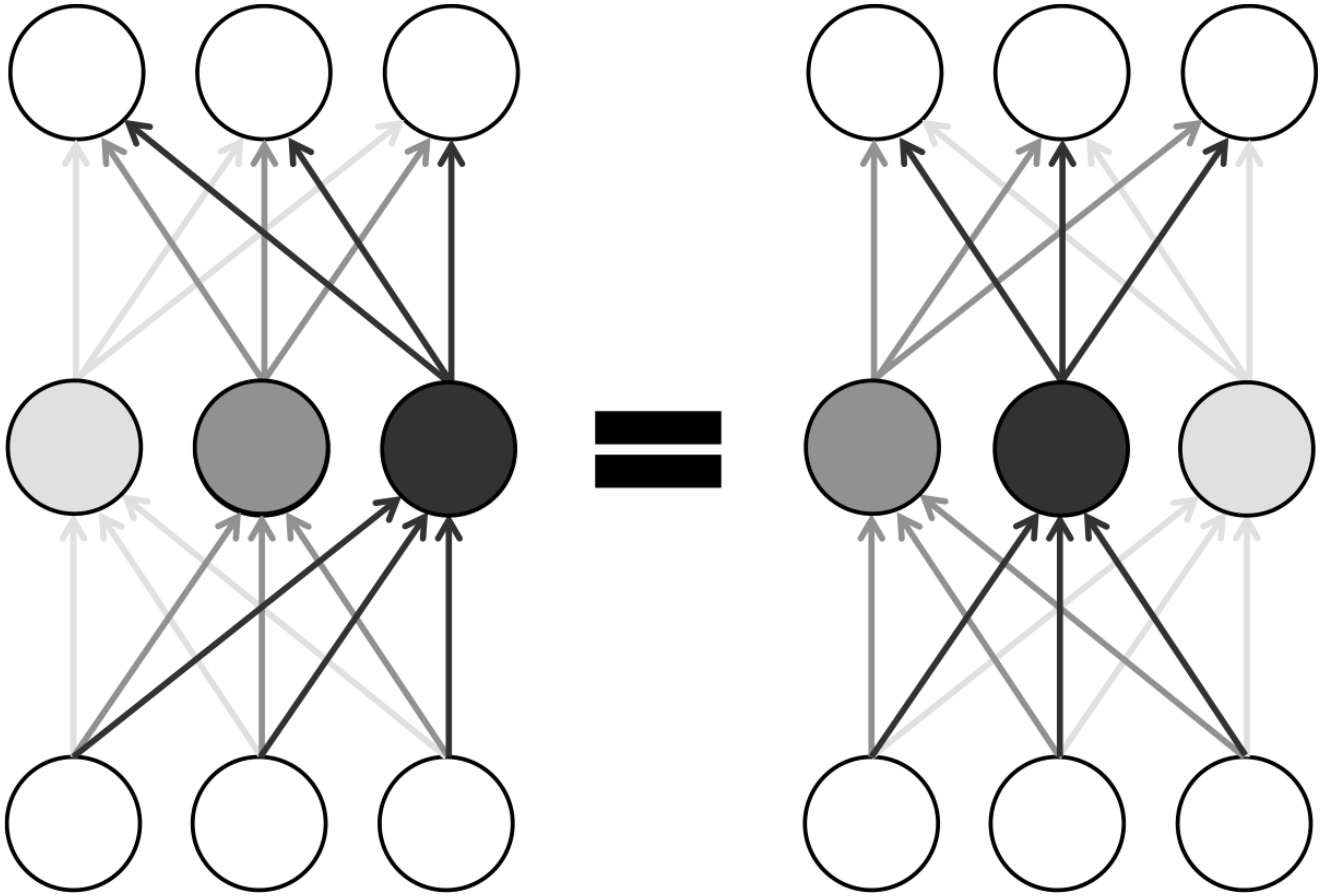
Neural Networks are not identifiable

First source of local minima

- within a layer with n neurons
  - there are $n!$ ways to rearrange parameters
- for a deep network with l layers, each with n neurons
  - we have a total of $n!^{l}$ equivalent configurations

Rearranging neurons

In a layer of a neural network results in equivalent configurations due to symmetry



Model Non-Indentifiability

- Second source of local minima
- For eg. ReLU neuron
  - ReLU uses a piecewise linear function
  - Infinite number of equivalent configurations

- No change in the behavior of the network
  - Multiplying all of the incoming weights by any nonzero constant *k*
  - *S*caling all of the outgoing weights by 1/*k*

Local minima are not problematic

- Caused due to non-identifiability characteristic of neural network
- As nonidentifiable configurations behave in an indistinguishable fashion no matter what input values are
- i.e. they will achieve the same error on the training, validation, and testing datasets
- All of these models will have learned equally from the training data
- will have identical behavior during generalization to unseen examples

Local minima are only problematic

- When they are spurious
- It corresponds to a configuration of weights in a neural network that incurs a higher error
- Gradient based optimization methods will be a failure

How Pesky Are Spurious Local Minima in Deep Networks?

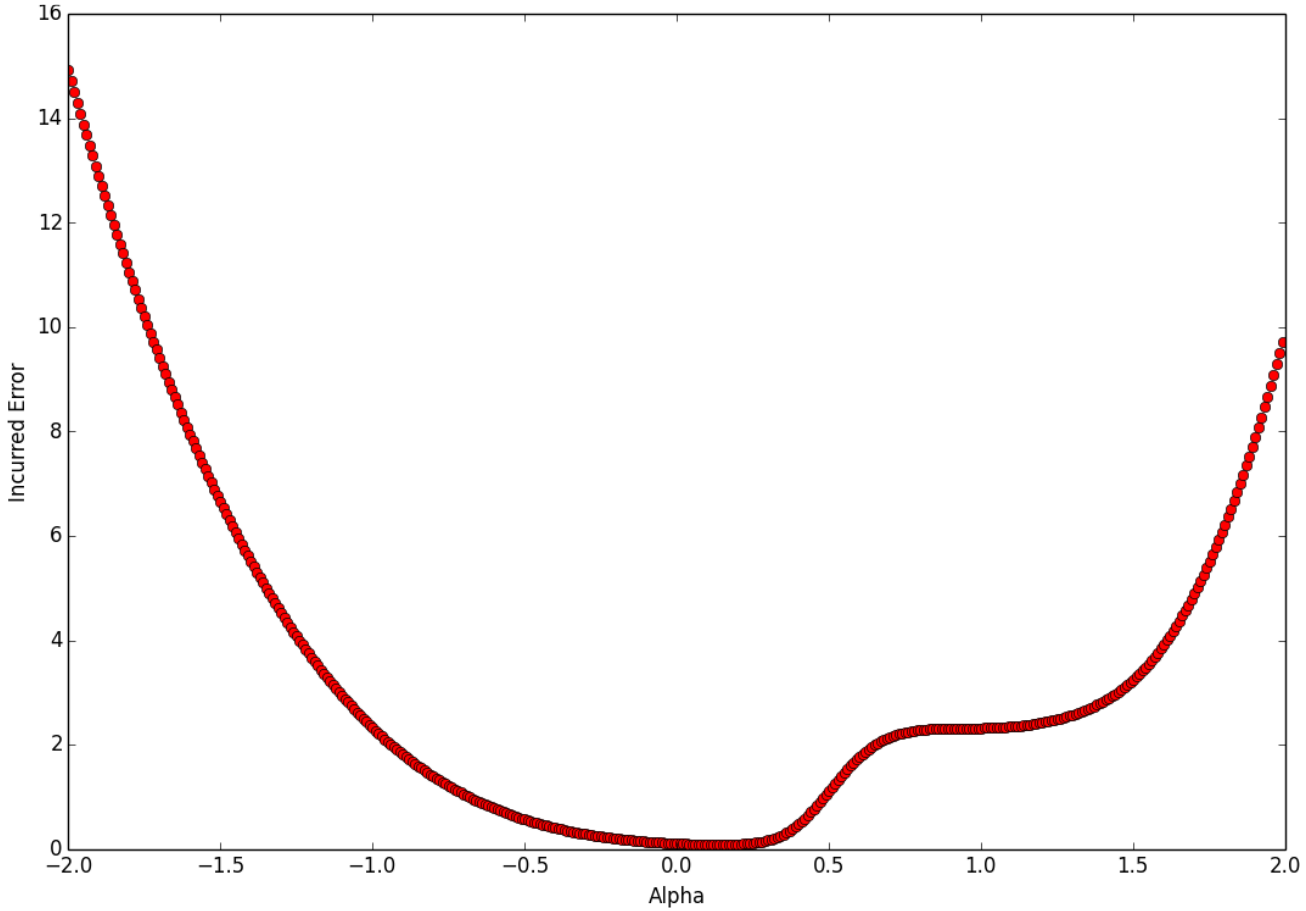
- Local minima have error rates and generalization characteristics that are very similar to global minima
- Plotting the error values does not give enough information about the error surface
- Goodfellow et al. (a team of researchers collaborating between Google and Stanford) published a paper in 2014

- Instead of analyzing the error function over time
  - investigated what happens on the error surface between a randomly initialized parameter vector and a successful final solution
  - used linear interpolation.

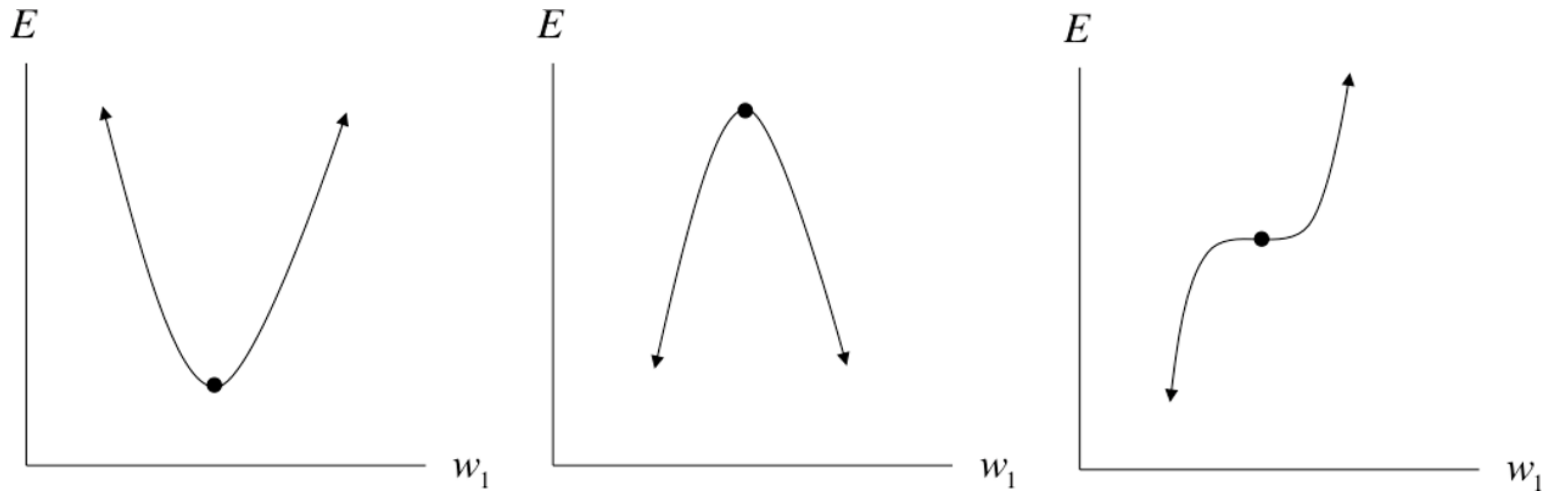
$$\theta_\alpha = \alpha \cdot \theta_f + (1 - \alpha) \cdot \theta_i.$$

Error function SGD randomly initialized parameter vector

- True struggle of gradient descent isn’t the existence of troublesome local minima
- But instead
  is that we have a
  tough time finding the
  appropriate direction to
  move in

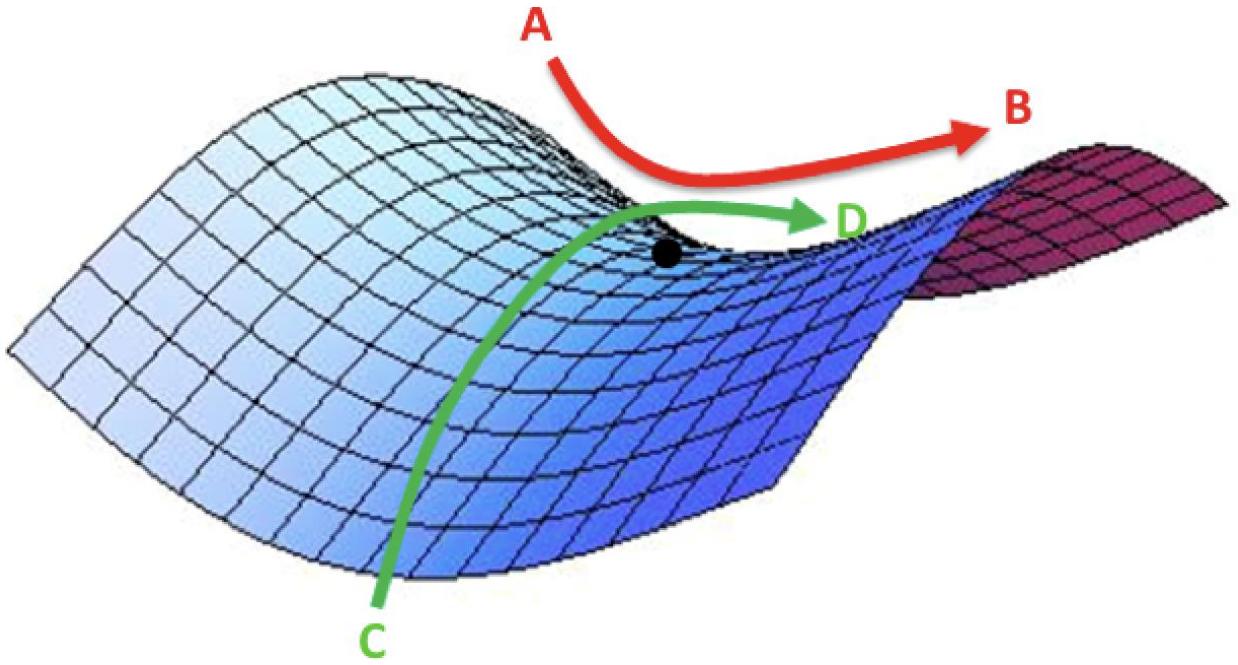


Flat regions in the error surface



- Assuming each of these three configurations is equally likely
- Given a random critical point in a random one-dimensional function
  - it has one-third probability of being a local minimum
  - if we have a total of *k* critical points, we can expect to have a total of *k*/3 local minima

- A random function with *k* critical points has an expected number of $k/3^d$ local minima
- In other words, as the dimensionality of our parameter space increases, local minima becomes exponentially more rare



Momentum Based Optimizations

- It computes an exponentially weighted average of gradients and then uses this gradient to update the weights as below.

$$v_t = \beta v_{t-1} + (1-\beta) dw_t$$

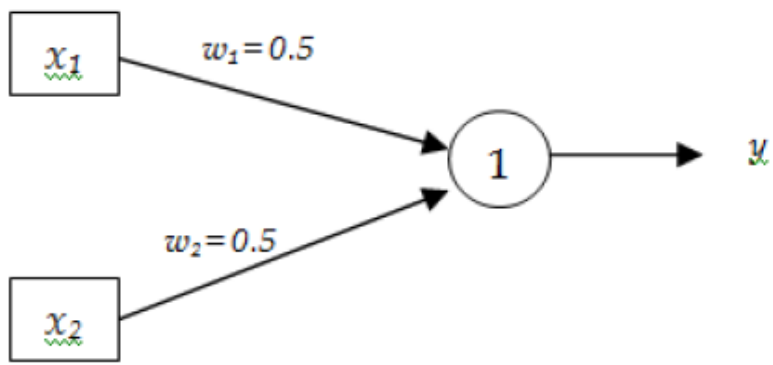
$$w_{t+1} = w_t + \alpha\, v_t$$

Where beta 'β' is Hyperparameter called momentum and ranges from 0 to 1.

Example

- Consider following simple ANN with logistic activation function. Assume that $\alpha = 0.2$ $\beta = 0.8$. Calculate weight updates for the given training Sample using Momentum.

x_1	x_2	t
1	0.6	0.8
0.5	0.5	0.6



Solution

For Training example:

(1,0.6,0.8)

$$y_{in} = 0.5*1 + 0.5*0.6 = 0.8$$

$$y = \frac{1}{1+\exp(-y_{in})} = 0.6899$$

$$(dw_1)_1 = (t-y)\times y\times(1-y)\times x_1 = 0.02354$$

$$(dw_2)_1 = (t-y)\times y\times(1-y)\times x_2 = 0.01412$$

$$(v_1)_1 = 0.8\times(v_1)_0 + 0.2\times 0.02354 = 0.004708$$

$$(v_2)_1 = 0.8\times(v_2)_0 + 0.2\times 0.01412 = 0.002824$$

$$(w_1)_2 = (w_1)_1 + 0.2\times(v_1)_1 = 0.5 + 0.2\times 0.004708 = 0.5009$$

$$(w_2)_2 = (w_2)_1 + 0.2\times(v_2)_1 = 0.5 + 0.2\times 0.002824 = 0.5006$$

Solution

For Training example:

(0.5,0.5,0.6)

$$y_{in} = 0.5009 * 1 + 0.5006 * 0.6 = 0.7512$$

$$y = \frac{1}{1 + \exp(-y_{in})} = 0.67944$$

$$(dw_1)_2 = (t - y) \times y \times (1 - y) \times x_1 = ?$$

$$(dw_2)_2 = (t - y) \times y \times (1 - y) \times x_2 = ?$$

$$(v_1)_2 = 0.8 \times (v_1)_1 + 0.2 \times (dw_1)_2 = ?$$

$$(v_2)_2 = 0.8 \times (v_2)_1 + 0.2 \times (dw_2)_2 = ?$$

$$(w_1)_3 = (w_1)_2 + 0.2 \times (v_1)_2 = ?$$

$$(w_2)_2 = (w_2)_2 + 0.2 \times (v_2)_2 = ?$$

- When V_0 *is* initialized to 0, then V_1 will be calculated as:

$$v_1 = 0.8v_0 + (1-0.8)x_1 = 0.2x_1$$

- Obviously, this is a rather inaccurate (too small) averaging for the weighted average of this day. Similarly, V_2 will be calculated as:

$$v_2 = 0.8v_1 + (1-0.8)x_2 = 0.8 \times 0.2 \times x_1 + 0.2x_2 = 0.16x_1 + 0.2x_2$$

- Again, this value is also inaccurate (too small).
- In order to correct this inaccuracy, you should use the formula:

$$v_t = \frac{v_t}{1-\beta^t} \quad where\ 1-\beta^t\ is\ Bias\ Correction\ Factor$$

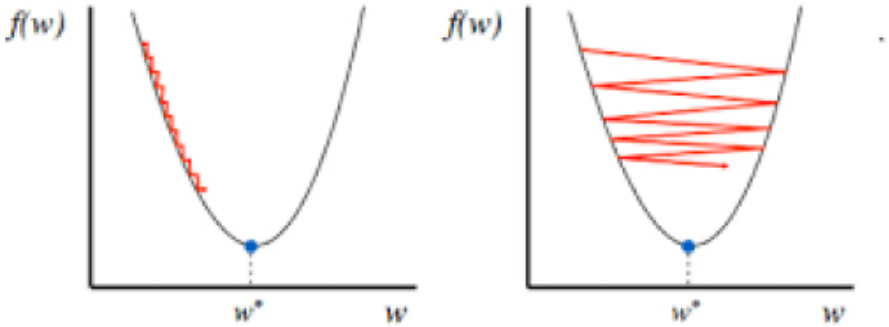
- Now v_1 and v_2 will be,

$$v_1 = \frac{v_1}{1-\beta^1} = \frac{v_1}{1-0.8^1} = \frac{v_1}{0.2} \qquad v_2 = \frac{v_2}{1-\beta^2} = \frac{v_2}{1-0.8^2} = \frac{v_2}{0.36}$$

- When t is sufficiently large, bias correction factor has no effect because $1/1-\beta^t \approx 1$. Hence It only jacks up the initial values.

Learning rate annealing

- One of the key hyper parameters to set in order to train a neural network is the *learning rate*.
- If learning rate is set too low, training will progress very slowly as we are making very tiny updates to the weights in our network.
- However, if learning rate is set too high, it can cause undesirable divergent behavior in our loss function.



- One approach of using learning rate value properly is to start with large value of learning rate and reduce value of the learning rate according to some predefined schedule. This approach is called learning rate annealing.
- Three commonly used learning rate annealing techniques or learning rate schedules are:
  - Time Based Decay
  - Step Decay
  - Exponential Decay

Time Based Decay

- Mathematical formulation of time-based learning rate decay is given below:

$$\alpha = \frac{\alpha_0}{1 + decay\_rate \times epoch\#}$$

Example

Let α_0=0.2 Decay_Rate=1

For Epoch=1 α=0.2/2=0.1

For Epoch=2 α=0.2/3=0.067

For Epoch=3 α=0.2/4=0.05

So on

Step Decay

- Mathematical formulation of step decay of learning rate is given below. A typical way is to drop the learning rate by half afterv every 10 epochs.

$$\alpha = \alpha_0 \times drop\_rate^{\lfloor epoch\#/epochs\_drop \rfloor}$$

Example

Let α_0=0.2 drop_rate=0.5 epochs_drop=5

For Epoch=5 $\alpha=0.2 \times 0.5^{5/5}=0.1$

For Epoch=10 $\alpha= 0.2 \times 0.5^{10/5}=0.05$

For Epoch=15 $\alpha= 0.2 \times 0.5^{15/5}=0.025$

Exponential Decay

- Mathematical formulation of exponential decay of learning rate is given below.

$$\alpha = \alpha_0 e^{(-k \times epoch\#)}$$

Example

Let α_0=0.2 k=0.1

For Epoch=1 α=0.181

For Epoch=2 α=0.164

For Epoch=3 α=0.148

Learning Rate Adaptation

Learning rate annealing techniques reduces learning rate equally for all parameters and applies the same learning rate with all parameters.

The better approach is to use the techniques that gives learning rate a chance to adapt.

The main idea behind adaptive learning rate is to use larger learning rate for updating parameters that are modified with small scale in past and use smaller learning rate for updating parameters that are modified with large scale in the past.

Adagrad

- In Adagrad, the variable v, called cache, keeps track of sum of the squared gradients, which in turn is used to normalize the parameter update.

$$v_t = v_{t-1} + dw_t^2$$

$$w_{t+1} = w_t + \frac{\alpha}{\sqrt{v_t + \varepsilon}} dw_t$$

- The effect of above equations is that Weights receiving high gradients will have their effective learning rate reduced.
- On the other hand, weights receiving small gradients will have their effective learning rate increased.

Adadelta

- Adadelta is an extension of Adagrad that seeks to reduce its aggressive, monotonically decreasing learning rate.
- Instead of accumulating all past squared gradients, Adadelta restricts the window of accumulated past gradients to some fixed size W.

RMSProp

- Adagrad decays the learning rate very aggressively (as the denominator grows). As a result, after a while, parameters receiving larger gradients will start receiving very small updates because of the decayed learning rate.
- To avoid this why not decay the denominator and prevent its rapid growth.
- RMSProp stands for root mean squared propagation and avoids monotonically increasing denominator of Adagrad.

RMSProp

- The central idea of RMSProp is keep the moving average of the squared gradients for each weight. And then divide the learning rate by root mean square of the squared gradients.

$$v_t = \beta v_{t-1} + (1-\beta) dw_t^2$$

$$w_{t+1} = w_t + \frac{\alpha}{\sqrt{v_t + \epsilon}} dw_t$$

Adam

- Adam update can be considered as RMSprop with momentum. In place of raw and noisy gradient vector *dw*, weighted average of gradients are used.

Weighted average of gradients

$$v_t = \beta_1 v_{t-1} + (1-\beta_1) dw_t$$

Weighted average of squared gradients

$$s_t = \beta_2 s_{t-1} + (1-\beta_2) dw_t^2$$

Adam

Then Bias Correction is applied to moving averages as below

$$v_t = \frac{v_t}{(1-\beta_1^t)} \qquad S_t = \frac{S_t}{(1-\beta_2^t)}$$

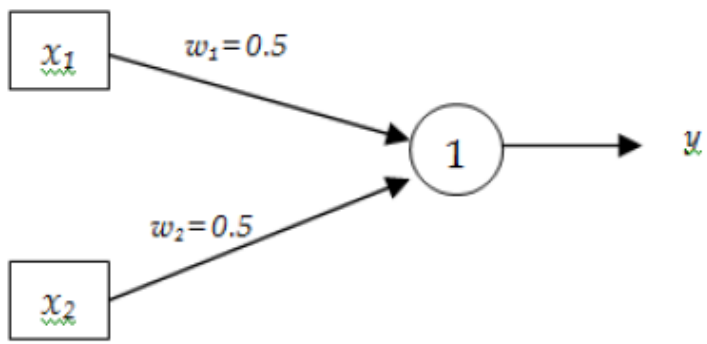
Finally, update weights as below:

$$w_{t+1} = w_t + \frac{\alpha}{\sqrt{s_t + \in}} v_t$$

Example

- Consider following simple ANN with logistic activation function. Assume that $\alpha = 0.2 \quad \beta = 0.8 \quad \beta_1 = 0.8 \quad \beta_2 = 0.8$. Calculate weight updates for the given training Sample using (1) Adagrad (2) RMSProp and (3) Adam

x_1	x_2	t
1	0.6	0.8
0.5	0.5	0.6
0.2	0.8	0.3



Adaptive Control of Learning Rate

Solution

For Adagrad

For Training example:

(1,0.6,0.8)

$$y_{in} = 0.5*1 + 0.5*0.6 = 0.8$$

$$y = \frac{1}{1+\exp(-y_{in})} = 0.6899$$

$$dw_1 = (t-y)\times y \times (1-y) \times x_1 = 0.02354$$

$$dw_2 = (t-y)\times y \times (1-y) \times x_2 = 0.01412$$

$$(v_1)_1 = (v_1)_0 + 0.02354^2 = 0.000554$$

$$(v_2)_1 = (v_2)_0 + 0.01412^2 = 0.000199$$

$$(w_1)_2 = (w_1)_1 + \frac{0.2}{\sqrt{0.000554}} \times (0.02354) = 0.7$$

$$(w_2)_2 = (w_2)_1 + \frac{0.2}{\sqrt{0.000199}} \times (0.01412) = 0.7$$

Solution

For Adagrad

For Training example: (0.5,0.5,0.6)

$$y_{in} = 0.7 * 0.5 + 0.7 * 0.5 = 0.7$$

$$y = \frac{1}{1 + \exp(-y_{in})} = 0.6682$$

$$dw_1 = (y - t) \times y \times (1 - y) \times x_1 = 0.007559$$

$$dw_2 = (y - t) \times y \times (1 - y) \times x_2 = 0.007559$$

$$v_1 = 0.000554 + 0.007559^2 = 0.000611$$

$$v_2 = 0.000199 + 0.007559^2 = 0.000257$$

$$w_1 = w_1 - \frac{0.2}{\sqrt{0.000611}} \times 0.007559 = 0.639$$

$$w_2 = w_2 - \frac{0.2}{\sqrt{0.000257}} \times 0.007559 = 0.606$$

Solution

For Adagrad

For Training example:

(0.2,0.8,0.3)

$$y_{in} = ????$$

$$y = \frac{1}{1+\exp(-y_{in})} = ??$$

$$dw_1 = (y-t)\times y\times(1-y)\times x_1 = ??$$

$$dw_2 = (y-t)\times y\times(1-y)\times x_2 = ??$$

$$v_1 = ??$$

$$v_2 = ??$$

$$w_1 = ??? = 0.531$$

$$w_2 = ??? = 0.412$$

Solution

For RMSProp

For Training example:(1,0.6,0.8)

$$y_{in} = 0.5*1 + 0.5*0.6 = 0.8$$

$$y = \frac{1}{1+\exp(-y_{in})} = 0.6899$$

$$dw_1 = (y-t)\times y\times(1-y)\times x_1 = -0.02354$$

$$dw_2 = (y-t)\times y\times(1-y)\times x_2 = -0.01412$$

$$v_1 = 0.8*0 + (1-0.8)*(-0.02354)^2 = 0.000111$$

$$v_2 = 0.8*0 + (1-0.8)*(-0.01412)^2 = 0.0000399$$

$$w_1 = w_1 - \frac{0.2}{\sqrt{0.000111}}\times(-0.02354) = 0.947$$

$$w_2 = w_2 - \frac{0.2}{\sqrt{.0000399}}\times(-0.01412) = 4.971$$

Solution

For RMSProp

For Training example:(0.5,0.5,0.6)

$$y_{in} = 0.947 * 0.5 + 4.971 * 0.5 = ??$$

$$y = \frac{1}{1 + \exp(-y_{in})} = ??$$

$$dw_1 = (y - t) \times y \times (1 - y) \times x_1 = ???$$

$$dw_2 = (y - t) \times y \times (1 - y) \times x_2 = ???$$

$$v_1 = 0.8 * 0.000111 + (1 - 0.8) * (dw_1)^2 = ????$$

$$v_2 = 0.8 * 0.0000399 + (1 - 0.8) * (dw_2)^2 = ???$$

$$w_1 = w_1 - \frac{0.2}{\sqrt{v_1}} \times (dw_1) = ???$$

$$w_2 = w_2 - \frac{0.2}{\sqrt{v_2}} \times (dw_2) = ???$$

Solution

For RMSProp

For Training example:(0.2,0.8,0.3)

$$y_{in} = ???$$

$$y = \frac{1}{1+\exp(-y_{in})} = ??$$

$$dw_1 = (y-t) \times y \times (1-y) \times x_1 = ???$$

$$dw_2 = (y-t) \times y \times (1-y) \times x_2 = ???$$

$$v_1 = 0.8 * ??? + (1-0.8) * (dw_1)^2 = ????$$

$$v_2 = 0.8 * ??? + (1-0.8) * (dw_2)^2 = ???$$

$$w_1 = w_1 - \frac{0.2}{\sqrt{v_1}} \times (dw_1) = ???$$

$$w_2 = w_2 - \frac{0.2}{\sqrt{v_2}} \times (dw_2) = ???$$